

Ultra-DSP: A Universal Lossless DSP Overpacking Framework for Low-Bit LLM Inference

Chenyu Zhang, Chenxi Xu, Jiaxiang Zou, Jingyu Guo, Rui Meng, and Xinyu Chen[†]

The Hong Kong University of Science and Technology (Guangzhou)

Guangzhou, China

{czhang539, cxu930, jzou521, jyguo012, rmeng603}@connect.hkust-gz.edu.cn

xinyuchen@hkust-gz.edu.cn

Abstract—Quantized LLM inference on FPGAs relies heavily on low-bit multiply-accumulate (MAC) operations. Yet DSPs, the main MAC engines in FPGAs, are built for high-precision computation and are therefore severely underutilized by low-bit arithmetic. Existing DSP packing techniques improve efficiency by packing multiple low-precision operations into one DSP, but they still cannot simultaneously achieve lossless high-density overpacking, universal support across precisions and DSP configurations, and efficient execution for both prefilling and decoding. In this paper, we propose Ultra-DSP, a high-density, lossless, and universal DSP overpacking framework for low-bit LLM inference. First, Ultra-DSP enables lossless high-density overpacking by decoupling signs from the packing domain and correcting overlap-induced MSB, LSB, and carry errors through coordinated boundary correction and LSB concatenation. Second, it introduces an overhead-aware Integer Linear Programming (ILP) solver that automatically generates efficient overpacking layouts across different precisions. Third, it builds a sharing-aware Adaptive Compute Engine (ACE) that supports both prefilling and decoding through hybrid packing-layout selection and shared correction hardware. Evaluations show that Ultra-DSP achieves $1.5\times$ – $2.25\times$ higher throughput than state-of-the-art DSP packing baselines, delivers a $2.22\times$ geometric-mean energy-efficiency gain over FlightLLM, and improves energy efficiency by up to $9.43\times$ over the GPU baseline on batch-1 GEMV workloads. The Ultra-DSP source code is available at <https://github.com/CLab-HKUST-GZ/Ultra-DSP-MICRO26-AE>.

I. INTRODUCTION

Transformer models, including large vision models and large language models (LLMs), achieve state-of-the-art performance across many applications [1]–[4], yet deployment demands enormous computation and memory bandwidth [5]–[8]. Low-bit quantization improves arithmetic density and memory efficiency [9]–[12]. INT4 weights require one quarter of FP16’s raw storage (a $4\times$ compression ratio), excluding quantization metadata. Recent advances substantially reduce low-bit accuracy loss, while 3-bit and 2-bit precisions are also actively explored [13]–[17].

FPGAs are an attractive platform for LLM acceleration because of their flexibility, customizability, and energy efficiency [5]–[8], [23]. Their main arithmetic engines are digital signal processing (DSP) blocks, which can execute arithmetic operations far more efficiently than LUT-based implementations [24], [25]. However, FPGA DSP blocks are provisioned

TABLE I
FEATURE COMPARISON OF DSP PACKING METHODS. STAGE:
P=PREFILLING, D=DECODING.

Methods	Data Type	Over-Packing	Lossless Precision	Universal Framework	Friendly Stage
WP486 [18]	INT8	✗	✗	✗	D
FlightLLM [7]	INT8	✗	✓	✗	D
WP521 [19]	INT4	✗	✗	✗	P
DB-MixQ [20]	INT4	✓	✗	✗	P
DSP-Packing [21]	INT4	✓	✗	✗	P
DuoQ [8]	INT4	✗	✓	✗	D
UDP [22]	Arbitrary	✗	✓	✗	P
Ultra-DSP	Arbitrary	✓	✓	✓	P+D

for much wider arithmetic (e.g., 27×18 -bit multiplication) than low-bit LLM inference typically requires, and are therefore severely underutilized by low-bit operations. As a result, many FPGA accelerators [26]–[29] map low-bit computation to LUTs and BRAMs instead, leaving valuable DSP resources underused. A natural way to improve DSP utilization is *DSP packing*, which consolidates multiple low-precision operations into a single DSP block.

Prior work has explored DSP packing extensively [19], [20], [22], [23]. Conventional *normal packing* [8], [18], [19], [22] avoids overlap between adjacent sub-results to preserve accuracy, but this strict isolation fundamentally limits packing density (e.g., only two INT8 or four INT4 multiplications per DSP). *Overpacking* methods [20], [21] relax this constraint by allowing sub-results to overlap, thereby increasing density (e.g., from 4 to 6 INT4 multiplications per DSP). However, they suffer from three major limitations. First, they are not lossless: overlap introduces numerical interference among adjacent sub-results, causing non-negligible accuracy degradation. Second, to support signed arithmetic, they rely on sign extension, which consumes precious input-port width and limits the achievable packing density. Third, existing methods do not provide a universal framework across diverse precisions, nor do they efficiently support the distinct prefilling and decoding phases of LLM inference.

To address these limitations, we propose **Ultra-DSP**, a high-density, lossless, and universal DSP overpacking framework for low-bit LLM inference. Ultra-DSP first removes sign-extension overhead from the packing domain by decoupling

[†]Corresponding author.

sign and magnitude, so that only magnitudes are densely packed into the DSP inputs. It then ensures exactness under aggressive overlap by correcting three coupled error components—MSB pollution, LSB pollution, and carry error—through coordinated boundary correction and final LSB concatenation. Building on this foundation, Ultra-DSP further introduces an overhead-aware Integer Linear Programming (ILP) solver that automatically generates efficient overpacking layouts across different precisions and DSP architectures. Finally, Ultra-DSP develops a sharing-aware Adaptive Compute Engine (ACE) that efficiently supports both prefilling and decoding through hybrid packing-layout selection and shared correction hardware. As shown in Table I, **Ultra-DSP** is the first framework that simultaneously provides losslessness, high packing density, generality, and phase-adaptive support for LLM inference.

Our main contributions are as follows:

- We propose a lossless high-density overpacking architecture that removes sign-extension overhead and corrects overlap-induced MSB, LSB, and carry errors with low-cost compensation, enabling exact low-bit DSP overpacking.
- We propose an overhead-aware universal ILP solver that replaces manual design and automatically generates optimized overpacking layouts across diverse precisions and DSP configurations.
- We propose a sharing-aware ACE for LLM inference, which supports efficient runtime switching between prefilling and decoding through hybrid packing-layout selection and shared correction logic.
- We conduct comprehensive evaluations. Compared with state-of-the-art DSP packing methods, Ultra-DSP achieves $1.5\times$ – $2.25\times$ higher throughput. At the system level, Ultra-DSP delivers a $2.22\times$ geometric-mean energy-efficiency gain over FlightLLM and up to $9.43\times$ higher energy efficiency than the GPU baseline on GEMV workloads.

II. BACKGROUND AND MOTIVATION

A. Quantized LLM Inference on FPGAs

Quantization reduces LLM inference costs by representing weights and activations as low-bit integers [8], [10], [11], [30]. Recent advances have moved designs from INT8 [31] toward increasingly explored 2–4-bit and sub-2-bit representations [11], [13], [14], [32], [33]. Figure 1 includes both weight-only and weight/activation settings, with INT4 offering an attractive accuracy–efficiency balance.

This shift makes efficient low-bit computation a first-order concern for FPGAs. Since DSP slices are the main arithmetic resource on modern FPGAs, supporting LLM inference at low precision requires extracting much higher MAC parallelism from each DSP slice. However, DSPs are fixed-function blocks designed for much higher precision. For instance, a DSP48E2 contains a fixed 27-bit pre-adder and a 27×18 -bit multiplier, as shown in Figure 2(a). Directly mapping INT4 or INT3 operations onto such units wastes most of their bit-level compute capacity, exceeding 80% in some cases (Figure 2(b)).

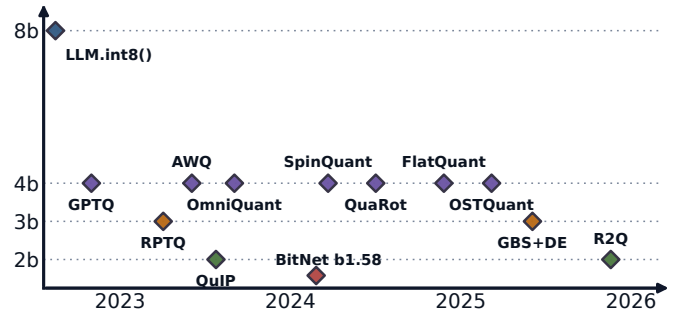
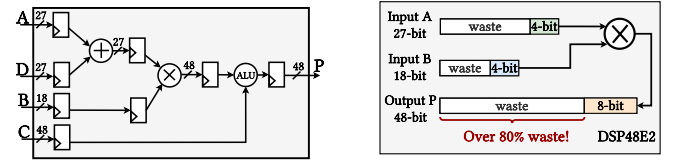


Fig. 1. Recent LLM quantization methods increasingly explore 2–4-bit weight-only and weight/activation precision settings.



(a) Schematic architecture of Xilinx (b) Waste under direct low-bit DSP48E2.

Fig. 2. DSP48E2 is designed for high-precision arithmetic, causing severe underutilization when directly mapped to low-bit LLM operations.

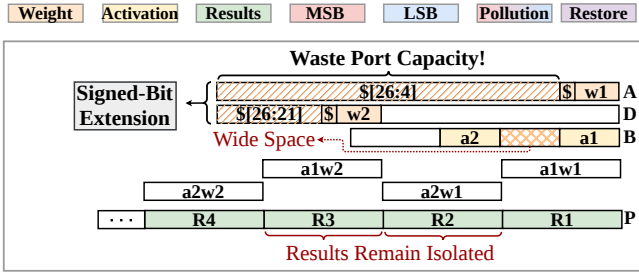
Therefore, densely packing multiple low-bit MACs into a single DSP slice is essential for efficient FPGA-based LLM inference workloads.

B. DSP Packing for Low-Bit Throughput

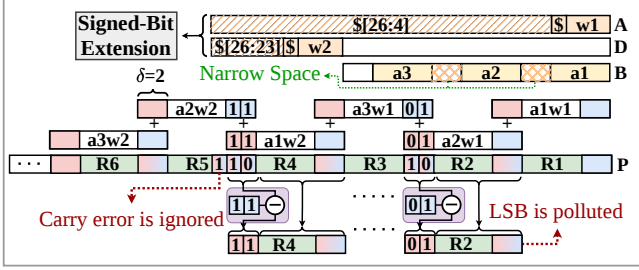
To mitigate the severe underutilization of DSP slices under low-bit arithmetic, prior work packs multiple low-bit operands into DSP input ports so that a single DSP can execute multiple multiplications in parallel. Existing methods generally fall into two categories: normal packing and overpacking.

Normal packing improves throughput by placing multiple low-bit values into the DSP input ports while strictly preventing overlap among adjacent partial results. Early studies such as Xilinx WP486 [18], Double MAC [34], and later WP521 [19] demonstrated that this approach can increase the number of low-bit multiplications supported by each DSP slice. Because adjacent sub-results must remain fully isolated at the output, sufficient spacing must also be reserved between neighboring operands at the input ports. As illustrated in Figure 3(a), products such as $a_2 \times w_1$ and $a_1 \times w_2$ must be strictly separated at output port P, which in turn limits how tightly operands can be packed into the DSP inputs. As a result, the achievable packing density of normal packing is fundamentally constrained by the isolation requirement.

Overpacking improves density by relaxing this strict isolation requirement. Instead of fully separating adjacent sub-results, it intentionally allows neighboring results to overlap by δ bits at the output, where δ denotes the number of shared bits between adjacent partial results. Increasing δ reduces the spacing required between neighboring operands in the input ports, thereby enabling higher packing density. As shown in Figure 3(b), DSP-Packing [21] adopts this strategy on



(a) Limitations of normal packing method.



(b) Limitations of overpacking method.

Fig. 3. Limitations of prior DSP packing methods. Normal packing wastes port capacity, while overpacking improves density but introduces overlap-induced numerical errors.

DSP48E2 in $(A + D) \times B$ mode, packing two signed weights into ports A and D and three unsigned activations into port B. With $\delta = 2$, adjacent partial results are compressed from 8 bits to 6 bits, allowing port B to accommodate three activations instead of two and enabling up to six INT4 multiplications in one DSP.

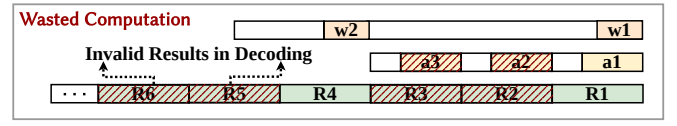
Overall, DSP packing substantially improves low-bit compute density on FPGAs. However, as we show next, existing packing methods still cannot fully satisfy the accuracy, flexibility, and phase-specific requirements of quantized LLM inference in practice.

C. Limitations of Prior DSP Packing

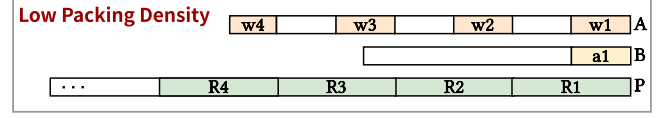
Although existing DSP packing methods improve low-bit throughput, they still fall short of modern LLM inference requirements in three aspects: exact computation, mixed-precision flexibility, and phase-adaptive execution.

Limitation 1: Lack of lossless high-density overpacking. Overpacking increases density by allowing adjacent sub-results to overlap. As shown in Figure 3(b), however, this overlap corrupts the MSBs, LSBs, and carry bits of neighboring results. Prior methods such as DSP-Packing [21] recover only part of this corruption, which leads to substantial accuracy degradation in downstream LLM tasks. In addition, signed arithmetic is usually handled through sign extension [8], [18]–[21], which consumes significant port width and limits packing density. Therefore, prior overpacking methods cannot simultaneously achieve exact computation and high density.

Limitation 2: Lack of a universal overpacking framework for mixed-precision LLM inference. LLM inference increasingly uses mixed-precision settings such as W4A4, W4A8, W2A8, and W8A8 across models and deployment settings. Ex-



(a) Prefilling-friendly packing strategy.



(b) Decoding-friendly packing strategy.

Fig. 4. Prior work lacks phase-adaptive support for LLM inference.

isting high-density schemes, however, are usually handcrafted for fixed precisions, while more general frameworks [22], [35]–[37] target normal packing and do not jointly optimize overpacking density and compensation overhead. As a result, prior work lacks a general overpacking framework that can support mixed-precision LLM inference.

Limitation 3: Lack of phase-adaptive packing for prefilling and decoding. LLM inference consists of two distinct phases. Prefilling exposes abundant parallelism and favors high-throughput packing, whereas decoding has limited parallelism and favors more efficient execution. Existing methods are usually optimized for a single operating point. As shown in Figure 4, prefilling-oriented schemes suffer severe utilization loss during the decoding phase, while decoding-oriented schemes sacrifice packing density and cannot meet prefilling throughput demands. Consequently, prior work cannot jointly optimize both phases for end-to-end LLM inference.

III. ULTRA-DSP OVERVIEW

Ultra-DSP addresses the key limitations of prior DSP packing methods for low-bit LLM inference through three tightly coupled components: (i) lossless high-density overpacking, (ii) an overhead-aware universal layout solver, and (iii) sharing-aware adaptive execution for prefilling and decoding.

Lossless High-Density Overpacking. Ultra-DSP improves signed packing density by excluding sign-extension overhead from the packing domain. It packs only unsigned magnitudes and introduces an exact recovery mechanism to remove overlap-induced errors, enabling lossless high-density signed overpacking in practice.

Overhead-Aware Universal Layout Solver. To support mixed-precision LLM inference, Ultra-DSP replaces handcrafted layouts with a universal solver. It formulates DSP overpacking as an overhead-aware ILP problem and automatically generates efficient layouts across precisions and DSP configurations, identifying practical operating points beyond density-only optimization.

Sharing-Aware Adaptive Engine for Prefilling and Decoding. Targeting shared-resource FPGA/edge accelerators in which one DSP array serves both phases, Ultra-DSP further adapts packing to the two phases of LLM inference. It derives phase-specific candidates for prefilling and decoding, then selects a sharing-aware hybrid design to maximize correction-logic reuse. Based on this design, Ultra-DSP builds ACE

to support runtime phase switching with shared datapath resources across both phases.

IV. HIGH-DENSITY LOSSLESS OVERPACKING

To achieve exact high-density overpacking, Ultra-DSP must solve two challenges: dense support for signed operands and exact recovery under aggressive overlap. Accordingly, we first remove sign-extension overhead from the packing domain, and then characterize and compensate the resulting overlap-induced errors.

A. Sign-Magnitude Decoupled Packing

The main bottleneck of direct signed overpacking is not arithmetic correctness, but the loss of input-port capacity caused by sign extension. As illustrated in Figure 6(a), directly packing signed operands wastes a large fraction of the A/D bit budget on replicated sign bits, leaving limited space for useful magnitudes. In contrast, Figure 6(b) shows that if signs are removed from the packing domain and handled separately, the same A/D budget can be used primarily for magnitudes, allowing more independent operands to be packed. This observation motivates the sign-magnitude decoupled packing strategy used throughout Ultra-DSP.

Specifically, Ultra-DSP decouples each signed two's-complement (S2C) operand into an explicit sign bit S and an unsigned magnitude through Sign-Magnitude Conversion. The sign bits are excluded from the packing domain, while only the magnitudes are placed on the DSP input ports. The sign of each product is then generated by XORing the operand signs. Meanwhile, by forcing the highest bit of each DSP input port to 0, the internal DSP multiplier operates purely in unsigned mode, completely eliminating sign extension inside the overpacked datapath. This unsigned multiplication stage produces only product magnitudes. Rather than carrying intermediate results in sign-magnitude form, Ultra-DSP restores the sign only at the output boundary through sign recovery and converts the corrected magnitude back to S2C using the XOR-derived result sign. This design keeps accumulation in the two's-complement domain and avoids sign-dependent add/subtract logic on intermediate results.

B. Error Decomposition and Compensation

1) *Error Analysis:* Aggressive overpacking improves density by shrinking the spacing between adjacent sub-results, but it also introduces boundary interference. Specifically, the least significant bits (LSBs) of a higher-order product can spill into the most significant bits (MSBs) of its lower-order neighbor, and the overlapped addition may further generate an out-of-boundary carry. Figure 5(b) illustrates this effect. At the right boundary, the LSBs of $R2$ ("11") overlap with the MSBs of $R1$ ("10"), producing "101". The shared field is therefore corrupted, while the overflow bit introduces an additional carry error into the non-overlapped region. Similarly, at the left boundary, the LSBs of $R3$ ("01") overlap with the MSBs of $R2$ ("10"), corrupting the boundary field between $R3$ and $R2$.

Therefore, overlap between adjacent sub-results introduces three coupled error components:

- **MSB pollution**, where the boundary MSB field of the lower-order product is corrupted;
- **LSB pollution**, where the true low-order bits of the higher-order product are lost in the overlap;
- **Carry error**, where the overlapped addition generates a redundant carry that propagates into the non-overlapped output bit region.

2) *Exact Recovery Strategy:* Exact recovery must address three coupled error components: the polluted boundary MSBs, the lost true low-order bits, and the redundant carry generated in the shared field. Among them, direct recovery of the polluted MSB field is the most expensive, because the boundary region is already the result of multiple bit-level partial products and their carry interactions. Ultra-DSP therefore avoids reconstructing the corrupted MSB field itself. Instead, it observes that the boundary corruption is caused by the overlapping low-order contribution from the neighboring higher-order product. As a result, MSB recovery can be transformed into subtracting this overlapping LSB contribution.

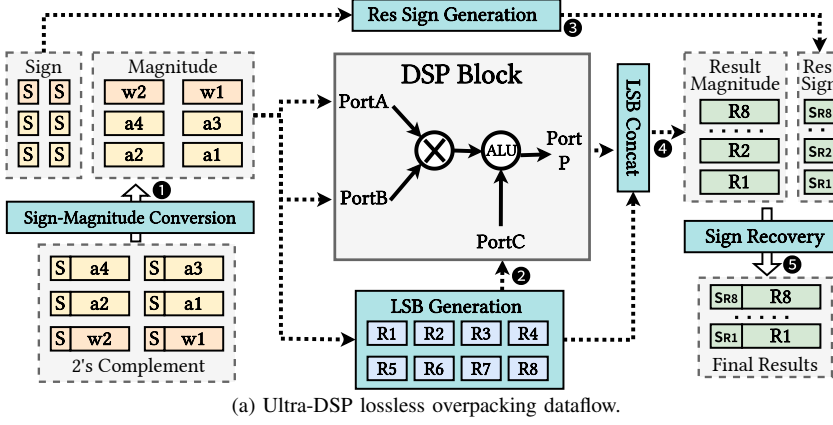
This subtraction also restores the correct carry behavior, because the redundant carry is generated by the same overlapping low-order term that corrupts the shared boundary field. The remaining error component, namely LSB pollution, is handled separately by directly concatenating the sub-result's own true low-order bits at the output. In this way, Ultra-DSP restores exactness through two coordinated operations: boundary correction via LSB subtraction, and true-LSB restoration via output concatenation.

Figure 5(c) illustrates this mechanism on a middle sub-result corrupted from both sides. At the left boundary, the polluted field "11" is corrected by subtracting the overlapping term "01", yielding the correct MSB field "10". At the right boundary, the polluted field "101" is corrected by subtracting the overlapping term "11", which removes the redundant carry and leaves the corrected upper bits "010". Finally, the true low-order bits "11" are concatenated with the corrected higher bits, reconstructing the exact result. In this way, Ultra-DSP eliminates MSB pollution, LSB pollution, and carry error in a unified recovery path.

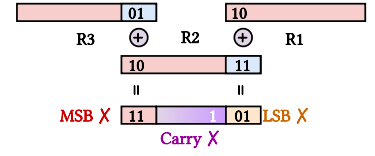
3) *Low-Cost Hardware Realization:* Ultra-DSP implements the recovery rule above with a compact LSB generator and the native DSP C port. The required correction terms are derived from the bit-level partial-product expansion $w_i \cdot a_j = \sum_{m=0}^{N-1} \sum_{n=0}^{M-1} (w_i[m] \wedge a_j[n]) \cdot 2^{m+n}$. The first few low-order bits are expressed as

$$\begin{aligned} (w_i a_j)[0] &= (w_i[0] \wedge a_j[0]) \\ (w_i a_j)[1] &= (w_i[0] \wedge a_j[1]) \oplus (w_i[1] \wedge a_j[0]) \\ (w_i a_j)[2] &= (w_i[2] \wedge a_j[0]) \oplus (w_i[1] \wedge a_j[1]) \\ &\quad \oplus (w_i[0] \wedge a_j[2]) \oplus ((w_i[1] \wedge a_j[0]) \wedge (w_i[0] \wedge a_j[1])) \end{aligned}$$

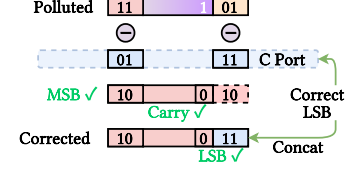
These Boolean expressions define the LSB generator used in Ultra-DSP. Since only a shallow low-order portion needs to be recovered, shallow LSB pre-computation is sufficient for the



(a) Ultra-DSP lossless overpacking dataflow.

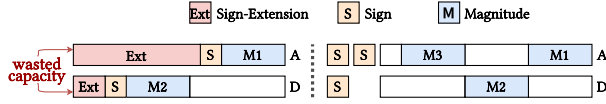


(b) Three types of overlapping error.



(c) Full error compensation.

Fig. 5. Ultra-DSP lossless overpacking dataflow and full error compensation.



(a) Direct packing with sign extension wastes port capacity. (b) Sign-isolated packing improves packing density.

Fig. 6. Motivation for sign-magnitude decoupled packing.

useful overlap range. For W4A4, synthesis shows the LUT cost increases from 1, 2, and 3 at depths 1–3 to 6, 11, and 17 at depths 4–6, while the marginal packing benefit quickly saturates. Ultra-DSP therefore limits the precomputed LSB depth to three bits as a practical balance between compensation effectiveness and implementation overhead. Figure 7 shows this knee is not W4A4-specific: across W3–W5 weights and A4/A5 activations, the cost stays at 1/2/3 LUTs through depth 3 and rises sharply beyond it.

At runtime, the generated LSB terms are aligned with the polluted bit positions and injected through the DSP C port. The internal ALU is configured in the $A \times B - C$ mode, so the DSP ALU subtracts these terms from the packed product to restore the polluted boundary and cancel the redundant carry. The same LSB terms are then forwarded to the output and concatenated back to recover the true low-order bits of the sub-result. Because the correction term enters through the C port of the same $A \times B - C$ DSP datapath, Ultra-DSP adds no separate post-multiply correction stage. The C-port register is pipeline-aligned, so correction does not add a separate stage or reduce throughput.

C. Lossless Overpacking Dataflow

Figure 5(a) summarizes the complete Ultra-DSP lossless-overpacking datapath by integrating the sign-decoupled packing and exact-recovery mechanisms. First, Sign-Magnitude Conversion ① decouples each signed two’s-complement operand into an explicit sign bit and an unsigned magnitude. The sign bits are sent to Res Sign Generation ③, while the magnitudes are packed onto Ports A/B and sent to LSB

Generation ② to pre-compute the intrusive low-order terms. Next, the DSP block performs overpacked multiplication, and the precomputed intrusive LSB terms are injected through Port C to cancel overlap-induced MSB and carry pollution. Finally, LSB Concat ④ restores the true low-order bits, and Sign Recovery ⑤ combines the corrected magnitudes with the precomputed result signs to reconstruct the exact final outputs.

D. Hardware-Efficient Realization

Although Ultra-DSP enables lossless high-density overpacking, a straightforward implementation still incurs nontrivial LUT overhead in sign conversion, intrusive-LSB delivery, and sign restoration. Ultra-DSP reduces these through three complementary hardware optimizations.

Offline weight conversion. Since weights are statically known, Ultra-DSP converts them from two’s-complement to sign-magnitude form offline and stores them directly in that format, eliminating the weight-side conversion logic from the runtime datapath.

Registered C-port driving. The precomputed intrusive LSBs are used for both in-situ compensation and final LSB concatenation. Driving the DSP C port directly from the combinational LSB generator creates a high-fanout path and often triggers logic replication. Ultra-DSP instead reuses the first register in the LSB delay chain to drive the C port, reducing fanout pressure and LUT duplication.

Accumulate-then-convert sign restoration. Converting each output magnitude back to two’s-complement form before accumulation is costly, since the $+1$ term of two’s-complement negation must be handled for every product. Ultra-DSP instead applies the following transformation:

$$V_{S2C} = (V_{Mag} \oplus S) + S, \quad \sum V_{S2C} = \sum (V_{Mag} \oplus S) + \sum S.$$

This structure restricts the main adder tree to accumulating only the XOR terms, while a lightweight side tree computes the sign sum. The two partial sums are then merged through a single final wide addition, replacing multiple per-product wide adders with one final adder and lightweight bit-level logic. To

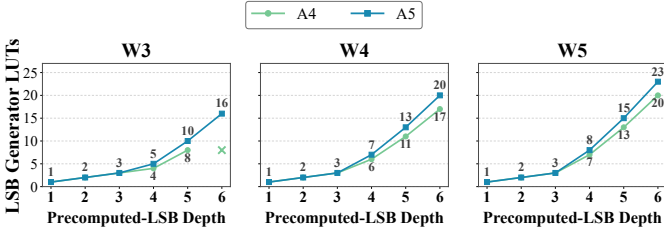


Fig. 7. LUT cost of precomputed-LSB generation across overlap depths and precisions.

quantify the non-DSP cost of lossless correction, we evaluate the cumulative LUT overhead of the three optimizations. In a representative W4A4 overpacking configuration, synthesis results show that they reduce this overhead from 121 to 108, 91, and 75, for a 38.0% overall reduction. Collectively, these optimizations keep the correction path lightweight enough to preserve Ultra-DSP’s practical density advantage.

V. OVERHEAD-AWARE UNIVERSAL LAYOUT SOLVER

While Section IV focuses on achieving exact high-density overpacking, a practical LLM inference engine must also support diverse mixed-precision settings across models and deployment settings. This creates a large design space of operand-width combinations and DSP packing layouts. Unlike normal packing, overpacking quality is determined not only by raw packing density, but also by the compensation overhead induced by overlap. As a result, a layout with higher density is not necessarily more hardware-efficient.

Therefore, for each mixed-precision overpacking configuration, Ultra-DSP must identify a layout that jointly balances legality, density, and compensation overhead. However, this is a challenging optimization problem because the placement of every packed input element is variable, causing the search space to grow exponentially. In particular, given packed element counts (n_x, n_y) and DSP input port widths (W_x, W_y) , the number of possible input placements is on the order of $O(W_x^{n_x} \cdot W_y^{n_y})$. Such a large design space makes exhaustive search impractical and prevents high-quality layouts from being found manually.

To address this challenge, Ultra-DSP formulates layout generation as an overhead-aware Integer Linear Programming (ILP) problem. This formulation answers three questions: how to represent an overpacking layout, which layouts are physically feasible under DSP constraints, and which feasible layout minimizes hardware overhead. These three questions correspond to the three core parts of the solver: the solution representation, the feasible solution space, and the corresponding optimization objective.

A. Formulation of Overpacking Layouts

To optimize an overpacking layout, we first need a representation that captures input placement while allowing the resulting output overlaps to be derived directly. Ultra-DSP adopts a coordinate-based representation, in which a layout is specified by the bit offsets of all packed input elements. Once these input coordinates are fixed, the output intervals of

all packed multiplications, and hence their overlap patterns, are uniquely determined.

Specifically, we describe an overpacking layout for input elements X_i and Y_j by the packed element counts (n_x, n_y) and their bit offsets $\{x_i\}_{i=1}^{n_x}$ and $\{y_j\}_{j=1}^{n_y}$, where each X_i has width W_X and each Y_j has width W_Y . Without loss of generality, we anchor the first elements at the origin, i.e., $x_1 = 0$ and $y_1 = 0$, and require monotonically increasing offsets, i.e., $x_i < x_{i+1}$ and $y_j < y_{j+1}$.

Element X_i occupies the bit interval $[x_i, x_i + W_X - 1]$, and Y_j occupies $[y_j, y_j + W_Y - 1]$. Each packed multiplication $R_{ij} = X_i \cdot Y_j$ has width $W_R = W_X + W_Y$ and occupies the output interval $I_{ij} = [x_i + y_j, x_i + y_j + W_R - 1]$.

The next step is to define the overlapping bits. To increase packing density, Ultra-DSP places $\{X_i\}_{i=1}^{n_x}$ close to one another, which causes their partial products to overlap with adjacent elements. For a fixed Y_j , the row of product results $\{R_{ij}\}_{i=1}^{n_x}$ therefore contains adjacent overlapping intervals $R_{i,j}$ and $R_{i+1,j}$. The number of overlapping bits is:

$$\begin{aligned} \alpha_i &= (x_i + y_j + W_R - 1) - (x_{i+1} + y_j) + 1 \\ &= W_R + x_i - x_{i+1}, \quad i \in [1, n_x - 1] \end{aligned} \quad (1)$$

Note that y_j cancels out in the final formula, indicating that the α_i overlap counts are identical across different Y_j .

The proposed overpacking method can recover bits corrupted by overlap between two adjacent result elements. Therefore, the only remaining overlap that must be considered is between the last element of one result row and the first element of the next, i.e., $R_{n_x,j}$ and $R_{1,j+1}$. The number of overlapping bits is defined as:

$$\begin{aligned} \beta_j &= (x_{n_x} + y_j + W_R - 1) - (x_1 + y_{j+1}) + 1 \\ &= W_R + x_{n_x} + y_j - y_{j+1}, \quad j \in [1, n_y - 1] \end{aligned} \quad (2)$$

In summary, an overpacking layout is specified by the input-coordinate sequences $\{x_i\}_{i=1}^{n_x}$ and $\{y_j\}_{j=1}^{n_y}$. These coordinates determine both the output intervals and the associated pairwise overlap counts:

$$\begin{aligned} \alpha_i &= W_R + x_i - x_{i+1}, & i \in [1, n_x - 1] \\ \beta_j &= W_R + x_{n_x} + y_j - y_{j+1}, & j \in [1, n_y - 1] \end{aligned} \quad (3)$$

B. Constraints and Overhead Modeling

Given the coordinate-based layout formulation above, we next define the legal solution space and model the compensation overhead induced by overlap.

Input non-overlap constraints. Input operands must not overlap with each other. If two packed input elements share a bit position, that bit would affect all associated products, leading to excessive and hard-to-manage compensation overhead. Therefore, Ultra-DSP allows overlap only in the *output* domain, while requiring the packed input elements to remain mutually disjoint:

$$x_{i+1} - x_i \geq W_X, \quad y_{j+1} - y_j \geq W_Y. \quad (4)$$

Port-width constraints. The spans of the packed inputs and outputs must fit within the DSP ports. Let W_A , W_B , and W_P

denote the widths of input port A/D, input port B, and output port P, respectively. Then the following constraints must hold:

$$\begin{aligned} x_{n_x} + W_X &\leq W_A, & y_{n_y} + W_Y &\leq W_B, \\ x_{n_x} + y_{n_y} + W_X + W_Y &\leq W_P. \end{aligned} \quad (5)$$

Bounded-overlap constraints. For practical deployment, overlap must also be limited by the cost of exact compensation logic. Ultra-DSP limits the overlap of each adjacent output pair using the following bound:

$$0 \leq \alpha_i, \beta_j \leq \delta_{\max}, \quad \delta_{\max} = 3.$$

Using the overlap definitions in Section V-A, this gives

$$\begin{aligned} 0 &\leq W_R + x_i - x_{i+1} \leq \delta_{\max}, & i &\in [1, n_x - 1], \\ 0 &\leq W_R + x_{n_x} + y_j - y_{j+1} \leq \delta_{\max}, & j &\in [1, n_y - 1]. \end{aligned} \quad (6)$$

Overhead model. In Ultra-DSP, compensation logic is driven by the number of overlapped bits that must be corrected. Therefore, the total overlap count provides a practical first-order proxy for hardware overhead. Since each intra-row overlap α_i appears in all n_y rows, while each cross-row overlap β_j appears once between adjacent rows, the total compensation cost can be modeled as

$$Z = n_y \sum_{i=1}^{n_x-1} \alpha_i + \sum_{j=1}^{n_y-1} \beta_j. \quad (7)$$

Substituting the definitions of α_i and β_j yields

$$\begin{aligned} Z &= n_y \sum_{i=1}^{n_x-1} (W_R + x_i - x_{i+1}) \\ &+ \sum_{j=1}^{n_y-1} (W_R + x_{n_x} + y_j - y_{j+1}) \\ &= n_y [(n_x - 1)W_R - x_{n_x}] \\ &+ [(n_y - 1)(x_{n_x} + W_R) - y_{n_y}] \\ &= (n_x n_y - 1)W_R - (x_{n_x} + y_{n_y}). \end{aligned} \quad (8)$$

This leads to an important insight: for a fixed overpacking configuration (n_x, n_y) , minimizing compensation overhead is equivalent to maximizing the spatial extent $(x_{n_x} + y_{n_y})$ under the legality constraints above.

C. ILP-Based Optimal Layout Solver

For a given DSP configuration (W_A, W_B, W_P) with input bit widths (W_X, W_Y) and packed element counts (n_x, n_y) , the goal is to find the legal overpacking layout with minimum compensation overhead. As shown in Section V-B, for fixed (n_x, n_y) , minimizing the total overhead Z is equivalent to maximizing the spatial extent $(x_{n_x} + y_{n_y})$. The optimal layout can therefore be obtained by solving:

$$\begin{aligned} \max \quad & x_{n_x} + y_{n_y} \\ \text{s.t.} \quad & \begin{cases} (n_x - 1)W_X \leq x_{n_x} \leq W_A - W_X, \\ (n_y - 1)W_Y \leq y_{n_y} \leq W_B - W_Y, \\ x_{n_x} + y_{n_y} \leq W_P - W_X - W_Y, \\ (n_x - 1)W_R - x_{n_x} \leq \delta_{\max}(n_x - 1), \\ (n_y - 1)(x_{n_x} + W_R) - y_{n_y} \leq \delta_{\max}(n_y - 1). \end{cases} \end{aligned} \quad (9)$$

All constraints and the objective are linear, and the optimization variables are integers. Therefore, Equation 9 is an Integer Linear Programming (ILP) problem and can be solved efficiently by standard ILP solvers.

Once the optimal spatial extent (x_{n_x}, y_{n_y}) is obtained, the complete layout can be constructed by evenly distributing the available offsets across the packed elements:

$$\begin{aligned} x_i &= (i - 1) \cdot \left\lfloor \frac{x_{n_x}}{n_x - 1} \right\rfloor + \max(0, i - n_x + r_x), \\ y_j &= (j - 1) \cdot \left\lfloor \frac{y_{n_y}}{n_y - 1} \right\rfloor + \max(0, j - n_y + r_y), \end{aligned} \quad (10)$$

where

$$r_x = x_{n_x} \bmod (n_x - 1), \quad r_y = y_{n_y} \bmod (n_y - 1). \quad (11)$$

Here, the remainder terms r_x and r_y distribute the residual offsets to the last few elements, ensuring that the constructed layout exactly spans the optimal ranges x_{n_x} and y_{n_y} .

To identify the best overpacking layout for a given precision setting, Ultra-DSP enumerates all feasible packing counts:

$$n_x \leq \left\lfloor \frac{W_A}{W_X} \right\rfloor, \quad n_y \leq \left\lfloor \frac{W_B}{W_Y} \right\rfloor. \quad (12)$$

For each candidate (n_x, n_y) , the solver returns the overhead-optimal layout. The computational parallelism of one DSP is computed as follows:

$$P = n_x \cdot n_y,$$

while the corresponding overhead is approximated by Z . By jointly considering DSP cost per multiplication $\frac{1}{n_x n_y}$ and normalized logic cost $\frac{Z}{n_x n_y}$, Ultra-DSP obtains the Pareto frontier of legal overpacking layouts for the target data format.

This formulation allows end users to select the most suitable overpacking layout for different deployment goals, such as maximizing throughput, improving energy efficiency, or balancing LUT and DSP usage for higher overall FPGA resource utilization across deployment targets.

VI. ADAPTIVE COMPUTE ENGINE FOR PREFILLING AND DECODING

LLM inference contains two distinct phases: throughput-oriented prefilling and latency-critical decoding. Existing DSP packing layouts are typically optimized for only one of them. Prefilling-friendly layouts achieve high density but waste computation in decoding, while decoding-friendly layouts sacrifice prefilling density. Ultra-DSP therefore introduces ACE,

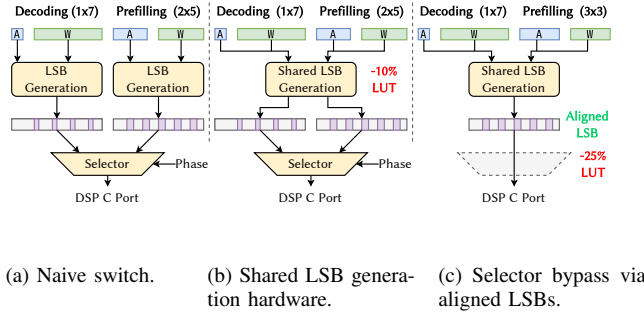


Fig. 8. Hardware overhead optimization for adaptive PE.

which supports both phases through phase-adaptive packing-layout selection and shared correction hardware. ACE targets FPGA/edge accelerators where prefilling and decoding time-share one limited computational array, as in same-device serving systems and FPGA/edge LLM accelerators under shared-resource constraints [5]–[7], [38]–[40].

A. Sharing-Aware Hybrid Packing-Layout Selection

A naive design independently selects the densest layout for each phase and instantiates two separate LSB-generation modules. As shown in Figure 8(a), each module produces its own correction pattern, and a phase-controlled selector routes the corresponding compensation vector to the DSP C port. This duplicates both LSB-generation and selection logic, leading to substantial overhead.

Ultra-DSP exploits the fact that different packing layouts share correction computation. The LSB-generation logic is reusable because the correction terms are derived from the same Boolean primitives. Rather than maintaining two independent generators, Ultra-DSP shares the LSB-generation hardware and keeps only phase-dependent routing of the correction terms, as shown in Figure 8(b). This removes duplicated generation logic and reduces the LUT cost of a PE by about 10%. A larger opportunity arises when the two layouts reconstruct LSBs at the same output bit positions. In this case, the correction paths align, the selector can be bypassed, and the compensation terms are sent directly to the DSP C port, as shown in Figure 8(c). This further reduces LUT usage by about 25%.

This observation changes the hybrid-selection objective. Rather than naively pairing the individually densest prefilling and decoding layouts, Ultra-DSP first uses the ILP solver to enumerate legal packing-count pairs (n_x, n_y) and extract two phase-specific Pareto candidate sets. It then performs an alignment-aware joint search over the Cartesian product of the two sets. For each layout pair, the key metric is the number of aligned LSB bits that can be shared across the two layouts, which directly captures the opportunity for correction-logic reuse. Since the search is restricted to phase-specific Pareto-optimal candidates, high throughput is already preserved within each phase; maximizing aligned LSB sharing therefore reduces hybrid overhead while retaining high throughput. Ultra-DSP thus selects the layout pair with the

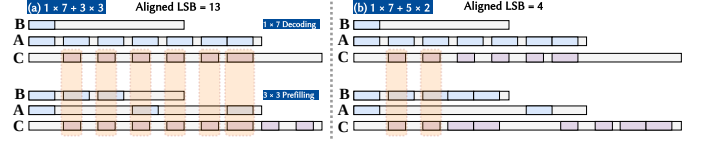


Fig. 9. Efficient hybrid support with shared compensation.

largest amount of shareable aligned LSB compensation, rather than simply combining the individually densest layouts.

Figure 9 illustrates this policy for W4A4. Although $5 \times 2 + 1 \times 7$ provides higher raw prefilling density than $3 \times 3 + 1 \times 7$, it shares only 4 aligned LSB-compensation bits with the decoding-optimal 1×7 layout, whereas 3×3 shares 13. Since both candidates come from the phase-specific Pareto sets, throughput is already preserved; the distinguishing factor is correction-logic reuse. Consequently, $3 \times 3 + 1 \times 7$ incurs much lower hybrid overhead (75 LUTs vs. 102 LUTs) while retaining high prefilling throughput (9 vs. 10 multiplications), and is therefore selected as the final hybrid packing layout.

B. Adaptive Compute Engine Architecture

We next describe how the selected hybrid layout pair is realized in hardware, from a single PE to the full ACE. To this end, we develop a parameterized Chisel [41] generator that emits a concrete Ultra-DSP PE for a target DSP variant and operand precision. The generated PE integrates the DSP primitive with the required compensation and sign-handling logic in a shared datapath across both phases. Instead of separate prefilling and decoding pipelines, runtime phase switching is realized through mode-dependent operand and output mappings around the same PE core.

Figure 10 shows the PE-level realization of the hybrid pair. A phase-mode signal reconfigures both input mapping and valid-output extraction around a shared DSP block with lossless overpacking. Under W4A4, prefilling instantiates the 3×3 layout, packing three activations and three weights and exposing all nine packed results, $R1$ – $R9$. Decoding switches to the 1×7 layout, enabling one activation and seven weights; the output mapping retains only the seven valid decoding results and suppresses the prefilling-only outputs. Thus, runtime adaptation changes only operand mapping and valid-output extraction, while reusing the DSP datapath and shared compensation logic.

Figure 11 shows how these generated PEs are integrated into the full ACE. ACE adopts a tile-parallel organization in which each tile contains multiple PEs, an on-chip activation buffer, and a memory controller connected to off-chip HBM channels. Weight data are streamed from HBM and dispatched to the corresponding PE lanes, while activations are buffered on chip and broadcast across the tile for reuse. A global PD-mode signal is distributed to all PEs in the tile so that the entire tile switches coherently between prefilling and decoding mappings at runtime. The resulting partial sums are accumulated along the tile datapath before write-back, and multiple ACE tiles can be instantiated in parallel to scale throughput across the available HBM channels.

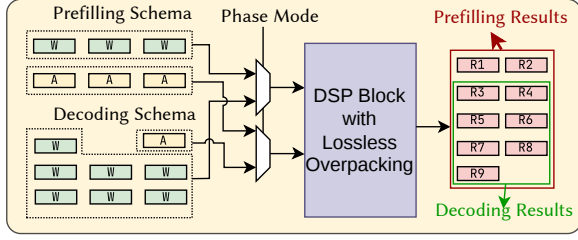


Fig. 10. Processing-element architecture of ACE.

VII. EVALUATION

A. Experimental Setup

Hardware implementation. We implement the proposed overpacking design in Chisel [41]. The generated Verilog is synthesized using Vivado 2023.2 and packaged as HLS black boxes for integration into ACE via Vitis 2023.2, targeting the Xilinx Alveo U55C. For kernel-level evaluation, ACE is configured as a 128×32 array operating at 200 MHz [5], [6]. For end-to-end LLM inference, we use a widely adopted open-source HLS framework for Llama-2-7B [42], replacing its default compute array with our proposed engine. To ensure fair model-level comparisons, only the MAC units in the compute array are replaced with prior DSP packing techniques, while all other dataflow and control logic remain unchanged.

Baselines. We compare Ultra-DSP against representative DSP packing methods, including WP521 [19], DSP-Packing [21], Deep-Burning-MixQ [20], and UDP [22], as well as the packing strategies used in recent FPGA-based LLM accelerators such as DuoQ [8] and FlightLLM [7]. We also compare ACE against optimized CPU and GPU implementations on an Intel Core i7-13700, an Intel Xeon Gold 6544Y, and an NVIDIA RTX 6000 Ada Generation GPU. The CPU baselines use Intel MKL `gemm_s8u8s32` [43], while the GPU baseline uses an optimized CUTLASS INT4 Tensor Core kernel [44].

Models and workloads. For model-level evaluation, we quantize Llama-2-7B and Llama-3-8B with OSTQuant [11], and report accuracy on four downstream tasks [45]–[48] using the lm-eval-harness framework [49].

B. Packing Density Evaluation

Table IIa reports the comparison between Ultra-DSP and prior DSP packing methods under standard precisions. We compare the number of packed multiplications per DSP (T) and the average DSP/LUT/FF cost per effective multiplication for both prefilling and decoding. The results show that Ultra-DSP achieves the highest effective arithmetic density in both phases. Under W4A4, it supports 9 packed multiplications per DSP in prefilling and 7 in decoding, whereas DSP-Packing [21], DuoQ [8], and UDP [22] achieve 6/2, 4/4, and 6/3, respectively. Relative to DSP-Packing, this corresponds to a **1.5 $\times$ improvement in prefilling and a 3.5 $\times$ improvement in decoding**. The additional LUT cost is a favorable trade-off, since such accelerators are typically constrained by DSPs rather than logic resources. Prior systems show the same

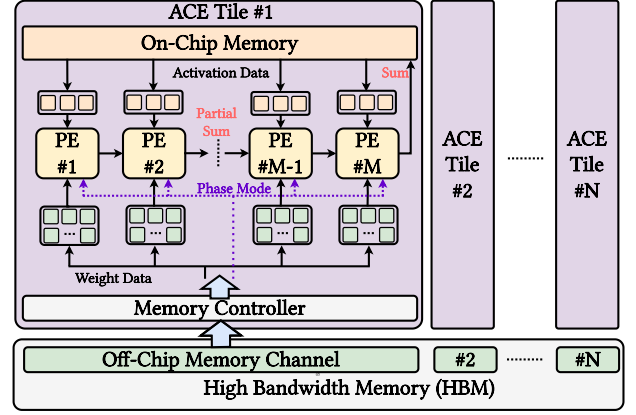


Fig. 11. Overview of ACE.

imbalance: DuoQ uses 90% of DSPs but only 37% of LUTs and 19% of FFs [8], while FlightLLM uses 70% of DSPs, 44% of LUTs, and 36% of FFs [7].

Table IIb shows the same analysis under mixed-precision settings. We compare Ultra-DSP with UDP [22] using the same metric across different precision combinations. We observe that Ultra-DSP consistently achieves higher packing density, **delivering a 1.5 $\times$ –2.0 $\times$ improvement in prefilling and a 2.5 $\times$ –4.5 $\times$ improvement in decoding over UDP**. The gain is more pronounced at lower precisions and becomes smaller as the operand bit-width increases. This is because lower-bit formats provide more packing headroom, whereas wider operands reduce packing opportunities and increase the logic cost per effective multiplication.

C. Kernel-Level Evaluation

We next evaluate whether the packing-density advantage translates to kernel-level gains on the proposed ACE. Following UDP [22], we remove platform-specific overheads such as HBM controllers, and retain only the compute units and essential I/O logic. For fairness, different packing schemes differ only in the MAC implementation, while the surrounding dataflow and control logic remain identical.

Compared with DSP packing methods. Table III compares Ultra-DSP with prior packing schemes under this controlled setup. We observe that Ultra-DSP achieves the highest throughput and energy efficiency across all five evaluated formats. Under W4A4, it reaches 14745.6 GOPS and 2371.44 GOPS/W, delivering **2.25 $\times$ higher throughput and 1.65 $\times$ higher energy efficiency** than WP521. Compared with UDP, Ultra-DSP improves throughput and energy efficiency by **1.50 $\times$ and 1.23 $\times$ under W4A4**, and by **1.50 $\times$ –2.00 $\times$ and 1.27 $\times$ –1.58 $\times$ across the mixed-precision formats**. The gain is larger at lower bit widths, where the higher packing potential allows Ultra-DSP to better exploit lossless overpacking. Although Ultra-DSP incurs additional LUT/FF cost and modestly higher power, the increase in delivered computation is larger, resulting in consistently higher GOPS/W across all evaluated low-bit data formats.

TABLE II
AVERAGE RESOURCE COST PER EFFECTIVE MULTIPLICATION FOR LLM PREFILLING AND DECODING. T DENOTES THE NUMBER OF EFFECTIVE MULTIPLICATIONS PACKED INTO ONE DSP. DSP, LUT, AND FF REPORT THE AVERAGE RESOURCE COST PER EFFECTIVE MULTIPLICATION.

(a) Comparison under standard precision settings.									
Method	Data Type	Prefilling				Decoding			
		T	DSP	LUT	FF	T	DSP	LUT	FF
WP486 [18]	W8A8	2	0.50	0.00	0.00	2	0.50	0.00	0.00
FlightLLM [7]	W8A8	2	0.50	7.00	8.50	2	0.50	7.00	8.50
WP521 [19]	W4A4	4	0.25	0.00	0.00	2	0.50	0.00	0.00
DB-MixQ [20]	W4A4	6	0.17	7.33	8.33	2	0.50	22.00	25.00
DSP-Packing [21]	W4A4	6	0.17	4.50	5.33	2	0.50	13.50	16.00
DuoQ [8]	W4A4	4	0.25	5.25	7.50	4	0.25	5.25	7.50
UDP [22]	W4A4	6	0.17	5.50	2.00	3	0.33	11.00	4.00
Ultra-DSP	W4A4	9	0.11	8.33	7.44	7	0.14	10.71	9.57

(b) Comparison under mixed-precision settings.									
Method	Data Type	Prefilling				Decoding			
		T	DSP	LUT	FF	T	DSP	LUT	FF
UDP	W3A4	6	0.17	7.50	2.50	2	0.50	22.50	7.50
Ultra-DSP	W3A4	12	0.08	8.08	7.25	9	0.11	10.78	9.67
UDP	W3A5	6	0.17	6.50	2.00	2	0.50	19.50	6.00
Ultra-DSP	W3A5	10	0.10	8.20	7.90	7	0.14	11.71	11.29
UDP	W4A5	4	0.25	9.00	2.75	2	0.50	18.00	5.50
Ultra-DSP	W4A5	8	0.13	10.25	7.88	6	0.17	13.66	10.50
UDP	W5A5	4	0.25	10.50	3.25	2	0.50	21.00	6.50
Ultra-DSP	W5A5	6	0.17	12.33	8.50	5	0.20	14.80	10.20

TABLE III
PERFORMANCE AND ENERGY-EFFICIENCY COMPARISON WITH EXISTING PACKING SCHEMES ACROSS DATA FORMATS.

Data formats	W4A4						W5A5		W4A3		W4A5		W5A3	
Work	WP521	DB-MixQ	DSP-Packing	DuoQ	UDP	Ultra-DSP	UDP	Ultra-DSP	UDP	Ultra-DSP	UDP	Ultra-DSP	UDP	Ultra-DSP
DSP	4096	4096	4096	4096	4096	4096	4096	4096	4096	4096	4096	4096	4096	4096
kLUT	12.05	99.27	106.74	98.34	148.83	244.94	150.85	172.12	131.56	273.40	119.13	253.55	153.19	226.81
kFF	28.59	205.65	54.45	158.93	80.01	259.99	60.95	119.54	61.93	321.96	62.98	234.64	58.65	287.27
Power (W)	4.57	4.90	4.98	5.45	5.08	6.22	4.81	5.28	4.79	6.63	4.78	6.06	4.83	6.35
GOPS	6553.60	9830.40	9830.40	6553.60	9830.40	14745.60	6553.60	9830.40	9830.40	19660.80	6553.60	13107.20	9830.40	16384.00
	(1.00×)	(1.50×)	(1.50×)	(1.00×)	(1.50×)	(2.25×)	(1.00×)	(1.50×)	(1.00×)	(2.00×)	(1.00×)	(2.00×)	(1.00×)	(1.67×)
GOPS/W	1433.11	2008.25	1974.37	1203.60	1933.60	2371.44	1361.36	1861.47	2050.56	2966.77	1370.47	2161.48	2034.44	2580.97
	(1.00×)	(1.40×)	(1.38×)	(0.84×)	(1.35×)	(1.65×)	(1.00×)	(1.37×)	(1.00×)	(1.45×)	(1.00×)	(1.58×)	(1.00×)	(1.27×)

¹ All GOPS measurements are conducted using the maximum packing configuration supported by DSP48E2.

Figure 12 evaluates the same-resource Pareto frontier with all methods capped at 4096 DSPs (45.4%) and 20%–45% LUT/FF budgets. The x-axis separates lossy baselines at 48–50% relative geomean accuracy from lossless designs at 100%. Ultra-DSP has the highest lossless prefill/decode throughput at every evaluated resource budget.

Compared with LUT-based methods. Table IV further compares Ultra-DSP with LUT-centric low-bit MAC implementations using normalized throughput per LUT (GOPS/kLUT), the key metric for logic-based designs. Ultra-DSP achieves the highest logic efficiency at **60.20 GOPS/kLUT**, compared with 2.42, 4.59, and 8.92 for Simple Mul., QSM Opt., and Radix-4 Opt., corresponding to at least a **6.75×** improvement in throughput per LUT. This advantage comes from mapping multiplication onto DSPs while using LUTs mainly for lightweight supporting logic, whereas LUT-centric designs implement low-bit multiplication directly in general-purpose logic and thus achieve much lower compute density per unit of logic.

Comparison with CPU and GPU. We further compare ACE against optimized CPU and GPU baselines, including Intel CPU and RTX 6000 Ada implementations. ACE is evaluated at the full-board level, streaming weights across 32 HBM channels. Table V shows that Ultra-DSP delivers the lowest latency and highest energy efficiency on GEMV workloads. For the $1 \times 2048 \times 2048$ GEMV, it achieves **37.92×** speedup and **61.07×** higher energy efficiency over the i7-13700, while remaining **2.41×** faster and **9.43×** more energy efficient than the RTX 6000 Ada. For the $1 \times 4096 \times 4096$ GEMV, it sustains **31.51×** speedup and **52.23×** higher energy

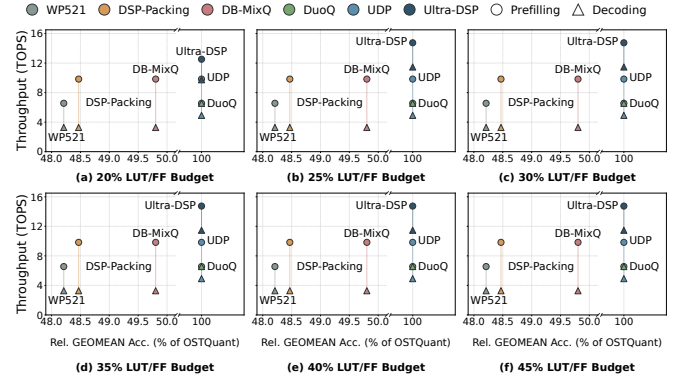


Fig. 12. Throughput–accuracy Pareto under different resource budgets. The x-axis reports accuracy relative to the original OSTQuant quantized baseline.

TABLE IV
COMPARISON WITH LUT-BASED APPROACHES.

Work	Method	DSP	kLUT	GOPS	GOPS/kLUT
UDP [22]	Simple Mul.	0	33.76	81.73	2.42
Cambricon-C [26]	QSM Opt.	0	17.91	82.29	4.59
LUTein [27]	Radix-4 Opt.	0	7.22	64.43	8.92
Ultra-DSP	Overpacking	4096	244.94	14745.60	60.20

efficiency over the i7-13700, and is still **1.05×** faster and **5.50×** more energy efficient than the GPU.

Figure 13 further compares Ultra-DSP against the GPU in the batched-GEMM regime. The CPU baseline is omitted from the figure because its throughput is much lower; at $B = 64$, the Xeon Gold 6544Y reaches only 3.31 TOPS

TABLE V
COMPARISON OF GEMV WORKLOAD PERFORMANCE ACROSS CPU, GPU,
AND FPGA PLATFORMS.

1 × 2048 × 2048 GEMV					
Platform	Time (ms)	Power (W)	Energy (mJ)	Speedup	Energy Eff.
MKL (i7-13700)	0.173	72	12.438	1.00×	1.00×
MKL (Xeon Gold 6544Y)	0.129	213	27.439	1.34×	0.45×
CUTLASS (RTX 6000 Ada)	0.011	175	1.924	15.72×	6.46×
Ultra-DSP (U55C)	0.005	45	0.204	37.92×	61.07×

1 × 4096 × 4096 GEMV					
Platform	Time (ms)	Power (W)	Energy (mJ)	Speedup	Energy Eff.
MKL (i7-13700)	0.574	75	42.850	1.00×	1.00×
MKL (Xeon Gold 6544Y)	0.369	229	84.325	1.56×	0.51×
CUTLASS (RTX 6000 Ada)	0.019	236	4.510	30.07×	9.50×
Ultra-DSP (U55C)	0.018	45	0.820	31.51×	52.23×

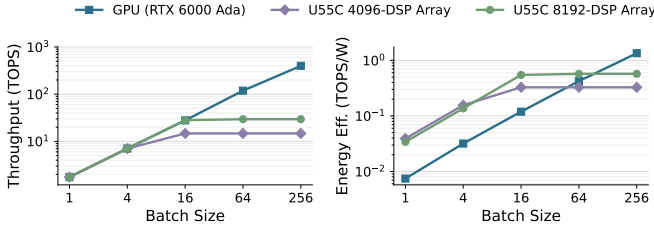


Fig. 13. Batched $[B, 4096] \times [4096, 4096]$ GEMM comparison on GPU and FPGA, where B is the batch size.

and 0.015 TOPS/W. The results show a batch-dependent split: RTX 6000 Ada dominates large-batch raw throughput because its INT4 peak is roughly $30\times$ the FPGA peak [50], [51]. In contrast, the FPGA remains more energy-efficient at small and moderate batches; for example, the 4096-DSP configuration achieves $2.74\times$ higher TOPS/W than the GPU at $B = 16$.

D. Model-Level Evaluation

We further evaluate end-to-end inference using an open-source HLS-based framework for Llama-2-7B [42]. All designs are deployed on an Alveo U55C at 200 MHz and evaluated across [prefilling length, decoding length] combinations. For fair comparison, all designs use the same framework, with only the MAC units in the compute array replaced by different packing schemes; memory bandwidth, nonlinear operators and dataflow remain unchanged.

End-to-End LLM Latency and Energy Efficiency. Figures 14(a) and 14(b) show that Ultra-DSP consistently achieves the best end-to-end performance among all baselines, with geometric-mean improvements of $2.31\times$ in speedup and $2.22\times$ in energy efficiency over FlightLLM [7]. The gain is larger in prefill-dominant cases such as $[1024, 128]$, where higher arithmetic density translates more directly into throughput, and smaller in decode-heavy cases such as $[512, 1024]$, where memory traffic becomes the primary bottleneck. Even in decoding, Ultra-DSP improves energy efficiency by reducing the number of active DSPs required to sustain the target throughput. Across all evaluated Llama-2-7B settings, Ultra-DSP achieves an average throughput of **98 tokens/s** and an energy efficiency of **0.56 tokens/J**.

Because the HLS framework [42] supports up to 4K tokens, the 8K study uses an extended ANT-based accelerator-level

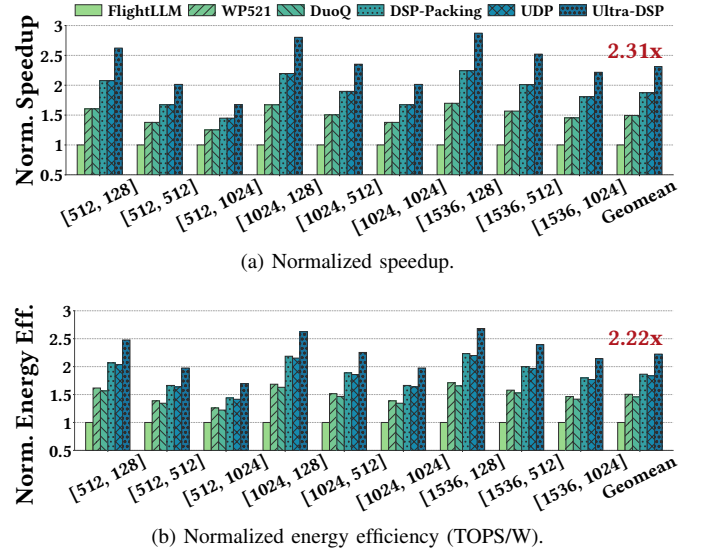


Fig. 14. End-to-end performance and energy efficiency across different sequence-length combinations. The horizontal axis denotes [prefilling length, decoding length].

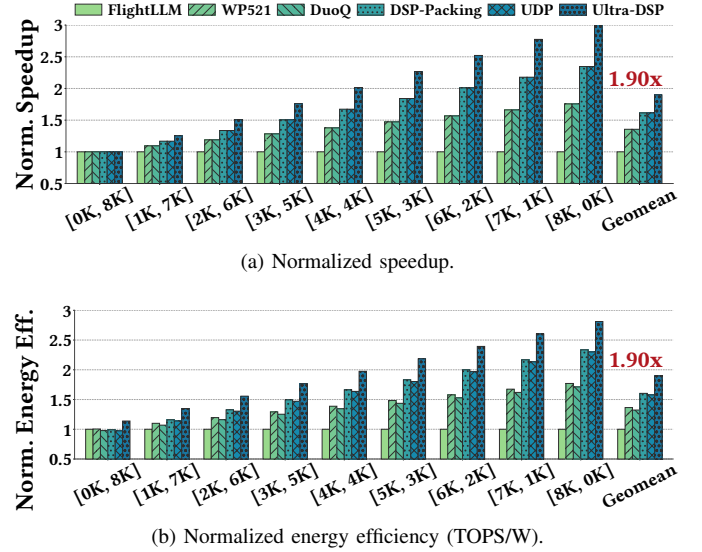


Fig. 15. End-to-end performance and energy efficiency across [prefilling, decoding] combinations totaling 8K tokens.

simulator [52], calibrated against measured shorter-context results. Figures 15(a) and 15(b) show that decode-heavy cases are memory-limited but still gain energy from fewer active DSPs, while prefill-dominant cases benefit from higher compute density. In pure prefilling $[8K, 0K]$, Ultra-DSP reaches $3.03\times$ speedup and $2.81\times$ energy efficiency; in pure decoding $[0K, 8K]$, it still improves energy efficiency by $1.14\times$.

Phase-adaptivity ablation. We isolate ACE by comparing against two fixed Ultra-DSP layouts. Relative to a fixed 3×3 prefill-friendly layout, ACE switches to 1×7 in decoding, nearly halves active DSPs, and improves end-to-end energy efficiency by $1.22\times$. Relative to a fixed 1×7 decode-friendly layout, ACE restores the 3×3 layout for prefilling and improves prefilling throughput by $1.28\times$.

End-to-End Model Accuracy. Table VI reports W4A4

TABLE VI
DOWNSTREAM TASK ACCURACY FOR LLM INFERENCE ACROSS
DIFFERENT OVERPACKING SCHEMES AT W4A4 PRECISION.

Model	Method	ARC-e	Hella	PIQA	OBQA	Avg
Llama-2-7B	Baseline (BF16)	74.49%	75.88%	79.00%	44.20%	68.39%
	Ultra-DSP	70.50%	73.26%	78.29%	41.80%	65.96%
	DSP-Packing	26.22%	26.05%	50.22%	25.40%	31.97%
	DB-MixQ	26.43%	26.35%	49.62%	29.00%	32.85%
Llama-3-8B	Baseline (BF16)	77.69%	79.22%	80.90%	45.00%	70.70%
	Ultra-DSP	74.83%	74.41%	78.18%	43.00%	67.61%
	DSP-Packing	25.67%	26.34%	52.67%	30.20%	33.72%
	DB-MixQ	23.61%	26.13%	51.57%	29.40%	32.68%

TABLE VII
GEOMEAN ACCURACY ACROSS MIXED W/A FORMATS OVER EVALUATED
TASKS AND MODELS.

Format	W5A5	W4A5	W4A4	W5A4	W3A5	W3A4	W5A3	W4A3
Ultra-DSP Geomean	68.86%	68.11%	66.78%	65.92%	63.81%	60.78%	45.94%	44.40%

accuracy on downstream tasks for both Llama-2-7B and Llama-3-8B. Ultra-DSP preserves high model quality on both models, reaching average accuracies of 65.96% and 67.61%, respectively, matching the original OSTQuant [11] accuracy under the same quantization setting. In contrast, prior overpacking schemes such as DSP-Packing and DB-MixQ suffer severe degradation, dropping to roughly 32–34%.

Table VII reports mixed-W/A geomean accuracy using the same models and downstream tasks as above. Because Ultra-DSP is bit-exact, it matches the original OSTQuant quantized baseline [11]. We also validate packing correctness independently of quantization. Using 1M random full-range two’s-complement operands per W/A pair, all 49 W2A2–W8A8 solver-selected layouts match an unpacked golden model exactly, with error probability (EP) and mean-squared error (MSE) both equal to zero.

E. Design Analysis

1) *ILP Solver Analysis*: Figure 16 shows the Pareto frontiers of solver-generated overpacking layouts from W3A4 to W5A5. The x-axis denotes normalized DSP cost per effective multiplication, and the y-axis denotes normalized compensation overhead per effective multiplication. Red and blue curves correspond to prefilling and decoding, respectively; gray points are dominated layouts, and stars mark the selected implementations. As precision decreases, the frontier moves toward the lower-left corner, indicating a better density–overhead trade-off. As bit width increases, it shifts rightward and upward because packing opportunities shrink and the per-MAC compensation cost increases. Under the same precision, prefilling consistently attains a better frontier than decoding, since its higher parallelism amortizes compensation overhead more effectively. These results show that the ILP solver efficiently identifies phase-specific Pareto-optimal operating points, with each frontier enumerated in less than one second on a consumer-grade CPU.

2) *Component Ablation*: Figure 17 separates Ultra-DSP’s gains from different optimization strategies. As shown in Figure 17(a), normal signed packing gives 4/2 products per

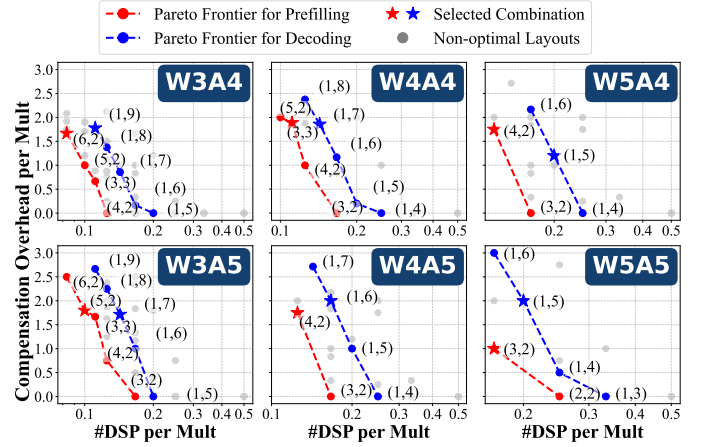
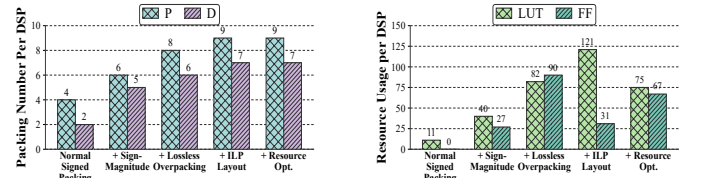
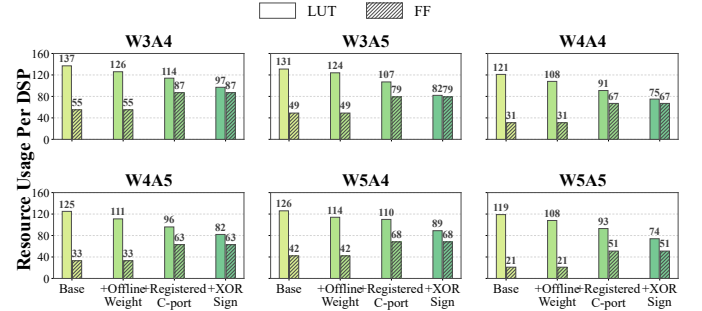


Fig. 16. Pareto frontiers of overpacking layouts under different bit-width settings. The lower-left corner indicates a better density–overhead trade-off.



(a) Component breakdown under W4A4.



(b) Cross-precision correction-overhead optimizations.

Fig. 17. Component and correction-overhead ablations.

DSP; sign-magnitude decoupling raises this to 6/5, lossless overpacking to 8/6, and ILP/resource optimization selects 9/7 at 75 LUTs/67 FFs. Figure 17(b) further shows that the three resource optimizations introduced in Sec. IV-D consistently reduce LUT usage by 29.2%–38.0% across W3A4–W5A5.

3) *Scalability Analysis*: Figure 18 shows that LUT, FF, and DSP usage grow approximately linearly as ACE scales from 256 to 8,100 PEs, indicating good implementation scalability and increasing throughput for compute-intensive workloads. At larger scales, routing pressure around the HBM interface, rather than compute-array inefficiency, reduces the maximum frequency from above 344 MHz at 1,024 PEs to 200–255 MHz. This post-route frequency remains within or above the 150–200 MHz range reported by prior end-to-end FPGA LLM accelerators [5], [6], [8]. Ultra-DSP therefore maintains practical timing closure at ACE scale without a severe frequency collapse.

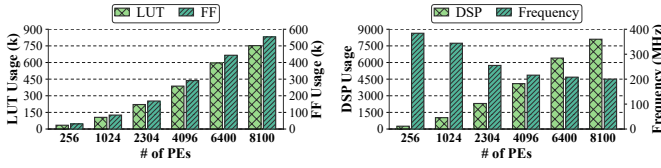


Fig. 18. Post-implementation resource usage and frequency of the ACE system with increasing numbers of PEs.

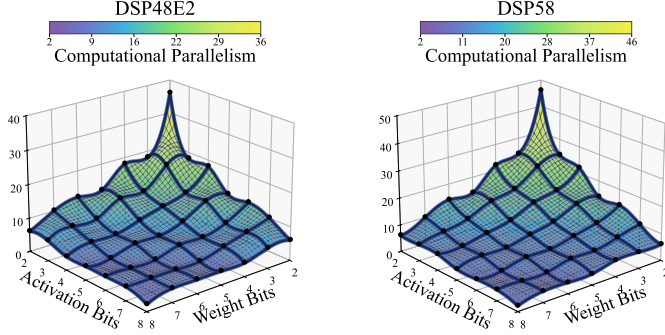


Fig. 19. Maximum parallelism per DSP48E2/DSP58 block across bit widths.

4) *Overlap-Depth Tradeoff*: In W4A4, depth 3 gives the best resource/timing tradeoff, packing 9/7 prefilling/decoding products per DSP with 75 LUTs/67 FFs per PE at 216 MHz. Depth 4 reaches 12/7 products but costs 157 LUTs/159 FFs and routes at 167.5 MHz. Depth 5 reaches 20/7 products, but its 310 LUTs/318 FFs per PE exceed U55C LUT capacity at the 4096-DSP scale.

5) *Generality Analysis*: Figure 19 reports Ultra-DSP’s maximum parallelism within a single DSP48E2/DSP58 block from W2A2 to W8A8. Ultra-DSP attains the highest density at W2A2, packing up to 36 multiplications in one DSP48E2 and 46 in one DSP58. These results indicate that Ultra-DSP is particularly effective in the low-bit regime targeted by quantized LLM inference.

Figure 20 reports computational efficiency across Xilinx DSP48E1/E2/DSP58 and Intel Stratix 10/Cyclone V, defined following UDP [22] as achieved parallelism normalized to normal packing’s theoretical maximum. Ultra-DSP exceeds $1.0\times$ in most configurations and peaks at $2.83\times$ for W2A2 on Intel DSPs and $2.71\times$ on DSP58, confirming its cross-architecture density advantage. For ASIC MACs without an FPGA DSP’s internal injection path, a small external adder performs compensation while preserving the remaining logic. TSMC 28nm synthesis for INT4 $\times$ INT4 shows **8/5** prefilling/decoding products on an INT16 $\times$ INT16 MAC with 18.25% area overhead and **16/9** on an INT32 $\times$ INT32 MAC with 16.7%, extending Ultra-DSP beyond FPGA DSP blocks.

Beyond text LLMs, Ultra-DSP can serve as the low-bit MAC backend for DSP-based ViT, CNN, and diffusion accelerators. Sparse high-precision outlier channels or tokens bypass the packed datapath and are accumulated separately. For FP/MX formats, a wrapper handles sign, exponent/block-scale metadata, normalization, rounding, saturation, and repacking; Ultra-DSP packs only decoded mantissa/magnitude products

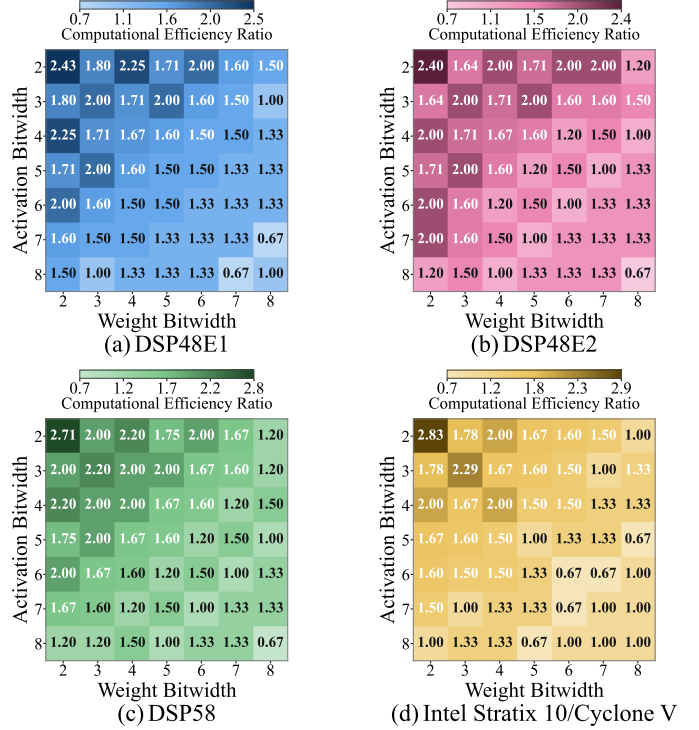


Fig. 20. Computational efficiency across different architectures and configurations.

TABLE VIII
MANTISSA BACKEND CAPACITY FOR LOW-BIT FP/MX FORMATS. T DENOTES MANTISSA PRODUCTS PER DSP.

Format	Backend T	LUT/DSP	Format	Backend T	LUT/DSP
MXFP4 E2M1	36	17	FP8 E4M3	10	16
FP4 E1M2	18	17	FP8 E3M4	6	4
FP8 E5M2	18	17	FP8 E2M5	4	6

and applies LSB correction. Table VIII reports product capacity and LUT cost for this scope.

VIII. CONCLUSION

This paper presents **Ultra-DSP**, a high-density and lossless DSP overpacking framework for low-bit LLM inference on FPGAs. Ultra-DSP addresses three key limitations of prior DSP packing methods. First, it enables lossless high-density overpacking by decoupling signs from the packing domain and restoring overlap-induced MSB, LSB, and carry errors through coordinated boundary correction and LSB concatenation. Second, it provides an overhead-aware universal layout solver that automatically generates efficient overpacking layouts across different precisions and DSP configurations. Third, it introduces a sharing-aware Adaptive Compute Engine (ACE) that supports both prefilling and decoding through hybrid packing-layout selection and shared correction hardware. Experiments demonstrate gains at both levels: up to **2.25 $\times$** higher kernel throughput; **2.31 $\times$** geometric-mean speedup and a **2.22 $\times$** energy-efficiency improvement over FlightLLM for end-to-end LLM inference; and up to **9.43 $\times$** higher energy efficiency than the GPU baseline on batch-1 GEMV workloads.

APPENDIX A

ARTIFACT APPENDIX

A. Abstract

The Ultra-DSP artifact provides the RTL, HLS sources, software models, experiment scripts, configuration files, and packaged evidence used to evaluate Ultra-DSP. It covers lossless signed low-bit DSP overpacking, layout and parallelism search, and phase-adaptive acceleration for prefilling and decoding. The artifact supports the evaluation of Tables II–VIII, Figures 12–13, and Figures 16–20 through arithmetic exactness tests, RTL simulation and synthesis, implementation-report parsers, analytical models, CPU/GPU measurements, model-accuracy workflows, and report validation. A hardware-free review path validates the packaged evidence without requiring an FPGA or GPU; optional workflows enable fresh CPU, GPU, RTL, synthesis, and model-accuracy experiments.

B. Artifact check-list (meta-information)

- **Algorithms/programs:** Lossless signed low-bit DSP overpacking, layout and parallelism search, phase-adaptive acceleration, Verilog RTL and testbenches, Vitis HLS kernels, Python analysis and verification scripts, and CPU/GPU reference kernels.
- **Compilation/transformations:** Python tools run after dependency installation. Optional FPGA workflows use Vivado and Vitis 2023.2; the GPU baseline uses CUDA-enabled PyTorch and CUTLASS.
- **Models/data:** Optional accuracy workflows use Llama-2-7B and Llama-3-8B with ARC-Easy, HellaSwag, PIQA, and OpenBookQA. Licensed model weights and third-party data are not redistributed.
- **Run-time environment:** Python 3.10 or newer with NumPy, SciPy, Matplotlib, Jupyter, and nbconvert for the portable analysis path. Platform-specific workflows additionally use Intel MKL, CUDA-enabled PyTorch, NVML, Vivado, Vitis HLS, and XRT.
- **Hardware:** Portable verification requires only a conventional x86-64 host. Optional platform-specific reproduction uses an AMD Alveo U55C, Intel Core i7-13700, Intel Xeon Gold 6544Y, NVIDIA RTX 6000 Ada, or an 80-GB-class CUDA GPU for full accuracy regeneration.
- **Execution/metrics:** Reported metrics include arithmetic error, LUT/FF/DSP utilization, frequency, throughput, latency, power, energy efficiency, perplexity, and zero-shot accuracy.
- **Output:** CSV/JSON summaries, logs, Vivado/Vitis reports, executed notebooks, and regenerated plots under `results/rerun/`.
- **Experiments:** Tables II–VIII, Figures 12–13 and 16–20, exactness, and RTL simulation.
- **Resources:** The archived artifact is approximately 26.3 MB, and an unpacked checkout requires well below 1 GB. Full accuracy regeneration additionally requires licensed model weights and tens of gigabytes for generated checkpoints and caches.
- **Preparation:** The portable Python workflow normally takes approximately 5–15 minutes to prepare after download. Vendor FPGA and CUDA environments require separate installation.
- **Experiment time:** Fast package checks complete within a few minutes. Synthesis and full-model experiments are hardware dependent; the recorded A800 training stage took approximately 23–24 minutes per model, excluding setup and evaluation.
- **Availability:** GitHub repository: <https://github.com/zcy-001-001/Ultra-DSP-MICRO26-AE>. Permanent archive: <https://doi.org/10.5281/zenodo.21485040>.

- **Code license:** The artifact is released under the Apache License, Version 2.0. Third-party models, data sets, and vendor tools retain their original licenses.
- **Workflow automation:** Workflows use Python, Bash, PowerShell, Tcl, and Jupyter nbconvert scripts.

C. Description

1) *How to access:* The public repository is available at <https://github.com/zcy-001-001/Ultra-DSP-MICRO26-AE>, and the permanent archive is available at <https://doi.org/10.5281/zenodo.21485040>. The evaluated GitHub tag and Zenodo release are both `tag-v1.0.2`. The top-level `README.md` describes the package layout, `REPRODUCE.md` provides table-by-table and figure-by-figure commands, and `results/RESULTS.md` maps the packaged outputs to the paper.

2) *Hardware dependencies:* No accelerator or proprietary tool is required for the package checker, analytical models, report parsers, and most plotting workflows. Fresh RTL and FPGA regeneration require Vivado/Vitis and target the AMD Alveo U55C. Optional baseline reproduction uses an Intel Core i7-13700, an Intel Xeon Gold 6544Y, or an NVIDIA RTX 6000 Ada. Full Table VI regeneration was validated with one NVIDIA A800-SXM4-80GB per model.

3) *Software dependencies:* Python 3.10 or newer is required. The portable analysis environment uses NumPy, SciPy, Matplotlib, Jupyter, and nbconvert. FPGA workflows require the 2023.2 FPGA toolchain (Vivado, Vitis, and Vitis HLS), XRT, and a compatible U55C platform file. CPU baselines use Intel MKL and RAPL access. The GPU baseline uses CUDA-enabled PyTorch, the bundled CUTLASS headers, and NVML through `pynvml`. The reference environment for Table VI is pinned in `results/table6/table6_fresh_a800_environment_lock.txt`.

4) *Data sets and models:* Licensed Llama model weights and evaluation data are not redistributed. The optional accuracy workflow expects reviewer-provided Llama-2-7B and Llama-3-8B model directories and obtains the evaluation tasks through the documented LM Evaluation Harness environment. Model and output locations are supplied through placeholders, so no machine-specific path is required.

D. Installation

For a GitHub clone, run the first command below; the Zenodo archive already contains the same fixed release and can begin with the second command.

```
git checkout tag-v1.0.2
python -m venv .venv
source .venv/bin/activate
python -m pip install numpy scipy matplotlib
python -m pip install jupyter nbconvert
python scripts/verify_artifact.py
```

On Windows, activate the environment with `.\.venv\Scripts\Activate.ps1`. Platform-specific dependencies and commands are documented in the corresponding experiment README files.

E. Experiment workflow

All commands are run from the repository root unless stated otherwise. Optional reruns write to `results/rerun/` and do not overwrite the packaged canonical evidence.

- **Fast review:** The package checker validates manifests, documentation, reports, summaries, provenance records, and expected results.
- **Exactness and RTL:** Run the 49-pair cross-precision arithmetic sweep and the six-case Vivado RTL workflow described in the top-level reproduction guide.
- **Paper results:** Regenerate or verify the analytical and report-derived results for Tables II–VIII, Figures 12–13, and Figures 16–20 using the item-specific commands in the same guide.
- **Fresh platform runs:** Optionally reproduce the CPU/GPU measurements, Vivado/Vitis synthesis, and Tables VI–VII accuracy experiments on matching hardware. Run the package checker again after completing any optional reruns.

F. Evaluation and expected results

A successful package check terminates with `ARTIFACT_VERIFY_PASS`. The exactness sweep produces 49 Ultra-DSP rows with `ep=0`, `mse=0`, and `error_count=0`. The RTL workflow prints `RTL_SIM_PASS cases=6`. The Figure 17 workflow reproduces the 75-LUT, 67-FF, one-DSP point and a rounded LUT-reduction range of 29.2%–38.0%.

The analytical FPGA model for Table V and Figure 13 uses 460 decimal GB/s memory bandwidth, 45 W, 200 MHz, and nine packed MACs per DSP per cycle, producing a batch-1 latency of 0.018236 ms. Table III and Figure 18 are verified by re-parsing the packaged implementation reports rather than requiring a new implementation run. For Table IV, the Ultra-DSP row is recomputed from packaged evidence, while the comparison rows remain literature-derived.

The package checker also validates provenance and scope for Figure 12, Tables VI–VIII, and the ILP notebooks for Figures 16, 19, and 20.

G. Experiment customization

Reviewer-owned model, result, and report paths, as well as the Vivado installation and U55C platform paths, are provided through placeholders documented in the top-level reproduction guide. Analytical sensitivity studies may vary bandwidth and DSP-count parameters. The exactness sweep exposes sample-count and random-seed controls, while the notebook and plotting workflows can be run independently. Customized runs should use a new subdirectory below `results/rerun/` so that packaged evidence remains immutable.

H. Methodology

The artifact review and badging process follows ACM guidance:

- ACM Artifact Review and Badging
- cTuning Artifact Evaluation

REFERENCES

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” 2021. [Online]. Available: <https://arxiv.org/abs/2010.11929>
- [2] Y. Bai, X. Geng, K. Mangalam, A. Bar, A. L. Yuille, T. Darrell, J. Malik, and A. A. Efros, “Sequential modeling enables scalable learning for large vision models,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 22 861–22 872.
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS ’20. Red Hook, NY, USA: Curran Associates Inc., 2020.
- [4] X. Dong, P. Zhang, Y. Zang, Y. Cao, B. Wang, L. Ouyang, X. Wei, S. Zhang, H. Duan, M. Cao, W. Zhang, Y. Li, H. Yan, Y. Gao, X. Zhang, W. Li, J. Li, K. Chen, C. He, X. Zhang, Y. Qiao, D. Lin, and J. Wang, “InternLM-XComposer2: Mastering free-form text-image composition and comprehension in vision-language large model,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.16420>
- [5] Y. Liang, H. Shi, H. Shao, and Z. Wang, “AccLLM: Accelerating long-context LLM inference via algorithm-hardware co-design,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.03745>
- [6] M. Huang, A. Shen, K. Li, H. Peng, B. Li, Y. Su, and H. Yu, “EdgeLLM: A highly efficient CPU-FPGA heterogeneous edge accelerator for large language models,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 7, pp. 3352–3365, 2025.
- [7] S. Zeng, J. Liu, G. Dai, X. Yang, T. Fu, H. Wang, W. Ma, H. Sun, S. Li, Z. Huang, Y. Dai, J. Li, Z. Wang, R. Zhang, K. Wen, X. Ning, and Y. Wang, “FlightLLM: Efficient large language model inference with a complete mapping flow on FPGAs,” in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, ser. FPGA ’24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 223–234. [Online]. Available: <https://doi.org/10.1145/3626202.3637562>
- [8] Z. Yu, H. Ji, Y. Cao, J. Wu, X. Yan, L. Zheng, and Z. Zou, “DuoQ: A DSP utilization-aware and outlier-free quantization for FPGA-based LLMs acceleration,” in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–7.
- [9] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefer, and J. Hensman, “QuaRot: Outlier-free 4-bit inference in rotated LLMs,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.00456>
- [10] Y. Sun, R. Liu, H. Bai, H. Bao, K. Zhao, Y. Li, J. Hu, X. Yu, L. Hou, C. Yuan, X. Jiang, W. Liu, and J. Yao, “FlatQuant: Flatness matters for LLM quantization,” 2025. [Online]. Available: <https://arxiv.org/abs/2410.09426>
- [11] X. Hu, Y. Cheng, D. Yang, Z. Xu, Z. Yuan, J. Yu, C. Xu, Z. Jiang, and S. Zhou, “OSTQuant: Refining large language model quantization with orthogonal and scaling transformations for better distribution fitting,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.13987>
- [12] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” 2023. [Online]. Available: <https://arxiv.org/abs/2210.17323>
- [13] Z. Yuan, L. Niu, J. Liu, W. Liu, X. Wang, Y. Shang, G. Sun, Q. Wu, J. Wu, and B. Wu, “RPTQ: Reorder-based post-training quantization for large language models,” *arXiv preprint arXiv:2304.01089*, 2023.
- [14] J. Chee, Y. Cai, V. Kuleshov, and C. De Sa, “QuIP: 2-bit quantization of large language models with guarantees,” in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS ’23. Red Hook, NY, USA: Curran Associates Inc., 2023.
- [15] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, “The era of 1-bit LLMs: All large language models are in 1.58 bits,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.17764>
- [16] J. Chen, J. Shi, J. Huo, and C. Wu, “R2Q: Towards robust 2-bit large language models via residual refinement quantization,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.21736>

- [17] L. Maisonnave, C. Moineau, O. Bichler, and F. Rastello, "Gradual binary search and dimension expansion: A general method for activation quantization in LLMs," 2025. [Online]. Available: <https://arxiv.org/abs/2504.13989>
- [18] Y. Fu, E. Wu, A. Sirasao, S. Attia, K. Khan, and R. Wittig, "Deep learning with INT8 optimization on Xilinx devices," Xilinx, Inc., Tech. Rep. WP486, April 2017, version 1.0.1.
- [19] T. Han, T. Zhang, D. Li, G. Liu, L. Tian, D. Xie, and Y. S. Shan, "Convolutional neural network with INT4 optimization on Xilinx devices," Xilinx, Inc., Tech. Rep. WP521, June 2020, version 1.0.1.
- [20] E. Luo, H. Huang, C. Liu, G. Li, B. Yang, Y. Wang, H. Li, and X. Li, "DeepBurning-MixQ: An open source mixed-precision neural network accelerator design framework for FPGAs," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–9.
- [21] J. Sommer, M. A. Ozkan, O. Keszocze, and J. Teich, "DSP-Packing: Squeezing low-precision arithmetic into FPGA DSP blocks," in *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*. Los Alamitos, CA, USA: IEEE Computer Society, Sep. 2022, pp. 160–166. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/FPL57034.2022.00035>
- [22] J. Wu, Z. Zheng, L. Gong, C. Wang, and X. Zhou, "UDP: A universal DSP packing framework for low-bitwidth MAC acceleration on FPGAs," in *Proceedings of the 2026 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, ser. FPGA '26. New York, NY, USA: Association for Computing Machinery, 2026, pp. 212–223. [Online]. Available: <https://doi.org/10.1145/3748173.3779194>
- [23] H. Wang, Y. Bai, J. Yu, and K. Wang, "TransFRU: Efficient deployment of transformers on FPGA with full resource utilization," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024, pp. 521–526.
- [24] S. Ullah, S. Rehman, M. Shafique, and A. Kumar, "High-performance accurate and approximate multipliers for FPGA-based hardware accelerators," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 2, pp. 211–224, 2022.
- [25] A. C. Mert, H. Azgin, E. Kalali, and I. Hamzaoglu, "Efficient multiple constant multiplication using DSP blocks in FPGA," in *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*, 2018, pp. 331–3313.
- [26] Y. Chen, Y. Zhao, Y. Hao, Y. Wen, Y. Dai, X. Li, Y. Liu, R. Zhang, M. Zou, X. Song, X. Hu, Z. Du, H. Chen, Q. Guo, and T. Chen, "Cambricon-C: Efficient 4-bit matrix unit via primitivization," in *Proceedings of the 2024 57th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '24. IEEE Press, 2024, pp. 538–550. [Online]. Available: <https://doi.org/10.1109/MICRO61859.2024.00047>
- [27] D. Im and H.-J. Yoo, "LUTein: Dense-sparse bit-slice architecture with radix-4 LUT-based slice-tensor processing units," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 747–759.
- [28] Y. Hao, Y. Zhao, C. Liu, Z. Du, S. Cheng, X. Li, X. Hu, Q. Guo, Z. Xu, and T. Chen, "Cambricon-P: A bitflow architecture for arbitrary precision computing," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 57–72.
- [29] J. Yang, Z. Zhang, Z. Liu, J. Zhou, L. Liu, S. Wei, and S. Yin, "FuseKNA: Fused kernel convolution based accelerator for deep neural networks," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 894–907.
- [30] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, and T. Blankevoort, "SpinQuant: LLM quantization with learned rotations," 2025. [Online]. Available: <https://arxiv.org/abs/2405.16406>
- [31] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2022.
- [32] J. Lin, J. Tang, H. Tang, S. Yang, G. Xiao, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," *GetMobile: Mobile Comp. and Comm.*, vol. 28, no. 4, pp. 12–17, Jan. 2025. [Online]. Available: <https://doi.org/10.1145/3714983.3714987>
- [33] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, "OmniQuant: Omnidirectionally calibrated quantization for large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2308.13137>
- [34] D. Nguyen, D. Kim, and J. Lee, "Double MAC: doubling the performance of convolutional neural networks on modern FPGAs," in *Proceedings of the Conference on Design, Automation & Test in Europe*, ser. DATE '17. Leuven, BEL: European Design and Automation Association, 2017, pp. 890–893.
- [35] J. Zhang, M. Zhang, X. Cao, and G. Li, "Uint-Packing: Multiply your DNN accelerator performance via unsigned integer DSP packing," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [36] X. Liu, Y. Chen, P. Ganesh, J. Pan, J. Xiong, and D. Chen, "HiKonv: High throughput quantized convolution with novel bit-wise management and computation," in *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2022, pp. 140–146.
- [37] M. Sun, Z. Li, A. Lu, Y. Li, S.-E. Chang, X. Ma, X. Lin, and Z. Fang, "FILM-QNN: Efficient FPGA acceleration of deep neural networks with intra-layer, mixed-precision quantization," in *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 134–145. [Online]. Available: <https://doi.org/10.1145/3490422.3502364>
- [38] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, "SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills," 2023. [Online]. Available: <https://arxiv.org/abs/2308.16369>
- [39] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen, "PowerInfer-2: Fast large language model inference on a smartphone," 2024. [Online]. Available: <https://arxiv.org/abs/2406.06282>
- [40] J. Li, T. Li, R. Chen, G. Shen, D. Zhao, Q. Zhang, and Y. Zeng, "Hummingbird: A smaller and faster large language model accelerator on embedded FPGA," 2025. [Online]. Available: <https://arxiv.org/abs/2507.03308>
- [41] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzyniek, and K. Asanović, "Chisel: Constructing hardware in a Scala embedded language," in *DAC Design Automation Conference 2012*, 2012, pp. 1212–1221.
- [42] A. He, D. Key, M. Bulling, A. Chang, S. Shapiro, and E. Lee, "HLSTransform: Energy-efficient Llama 2 inference on FPGAs via high level synthesis," 2024. [Online]. Available: <https://arxiv.org/abs/2405.00738>
- [43] Intel, "Intel oneAPI math kernel library (oneMKL) 2021 release notes," Dec. 2021, updated December 1, 2021. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/release-notes/onemkl-release-notes-2021.html>
- [44] V. Thakkar, P. Ramani, C. Cecka, A. Shivam, H. Lu, E. Yan, J. Kosaian, M. Hoemmen, H. Wu, A. Kerr, M. Nicely, D. Merrill, D. Blasig, A. Atluri, F. Qiao, P. Majcher, P. Springer, M. Hohnerbach, J. Wang, and M. Gupta, "CUTLASS," 2023, version 3.0.0, software. [Online]. Available: <https://github.com/NVIDIA/cutlass/tree/v3.0.0>
- [45] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have solved question answering? Try ARC, the AI2 reasoning challenge," *arXiv:1803.05457v1*, 2018.
- [46] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a machine really finish your sentence?" in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [47] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi, "PIQA: Reasoning about physical commonsense in natural language," in *Thirty-Fourth AAAI Conference on Artificial Intelligence*, 2020.
- [48] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal, "Can a suit of armor conduct electricity? A new dataset for open book question answering," 2018. [Online]. Available: <https://arxiv.org/abs/1809.02789>
- [49] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, "A framework for few-shot language model evaluation," Sep. 2021. [Online]. Available: <https://doi.org/10.5281/zenodo.5371628>
- [50] NVIDIA Corporation, "NVIDIA RTX 6000 Ada Generation Graphics Card," <https://www.nvidia.com/en-us/products/workstations/rtx-6000/>, 2026, accessed: 2026-06-16. The page reports 1457 AI TOPS Tensor Performance for NVIDIA RTX 6000 Ada; peak rates are based on GPU Boost Clock and the AI TOPS value is theoretical FP8 TOPS using sparsity.

- [51] D. K. Athur, R. Pawar, and A. Arora, "Out-of-the-box performance of FPGAs for ML workloads using Vitis AI," in *Applied Reconfigurable Computing. Architectures, Tools, and Applications: 21st International Symposium, ARC 2025, Seville, Spain, April 9–11, 2025, Proceedings*, ser. Lecture Notes in Computer Science, R. Giorgi, M. Stojilović, D. Stroobandt, P. Brox Jiménez, and Á. Barriga Barros, Eds., vol. 15594. Cham, Switzerland: Springer Nature Switzerland, 2025, pp. 123–139.
- [52] C. Guo, C. Zhang, J. Leng, Z. Liu, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "ANT: Exploiting adaptive numerical data type for low-bit deep neural network quantization," in *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '22. IEEE Press, 2023, pp. 1414–1433. [Online]. Available: https://doi.org/10.1007/978-3-031-87995-1_8